Topic:

Principal Component Analysis (PCA)

Course Title:

Machine Learning and Data Mining (CS6735)

Instructor:

Prof. Roozbeh Razavi-Far

Submitted By:

Joydip Das (3804528)

Refat Ishrak Hemel (3810002)

1 Introduction:

Principal Component Analysis (PCA) is a powerful linear dimensionality reduction technique used to reduce the number of features in a dataset while preserving as much information (variance) as possible. It transforms a set of correlated variables into a smaller set of uncorrelated variables called principal components.

2 Motivation:

The primary reason to use PCA is to combat the **curse of dimensionality**. As the number of features (dimensions) in a dataset increases, several issues arise:

- **Model Overfitting:** With too many features, a machine learning model may learn noise rather than patterns, leading to decreased accuracy on new data.
- **Performance Degradation:** More dimensions require more mathematical calculations, which slows down model training and inference.
- **Visualization Challenges:** Humans can only visualize data in up to three dimensions; PCA allows us to project high-dimensional data into 2D or 3D for better understanding.

3 Mathematical Foundations of PCA:

The idea behind PCA is to find a line that passes through the origin and captures the maximum variance and then the next maximum variance and so on. So, PCA's computational engine relies on understanding how linear transformations interact with vectors. In any coordinate shift, most vectors are knocked off their "span"—the line passing through their origin and tip. However, specific "special vectors" remain on their span, merely stretching or squishing without rotating. These are the **eigenvectors**, and they represent the stable directions of the data's transformation. And with the help of these eigenvectors and eigenvalues, the line can easily be found.

The relationship between a matrix A (the transformation) and its eigenvector v is:

$$Av = \lambda v$$

Here, λ (lambda) is the **eigenvalue**, the factor by which the vector is scaled. To solve for v, we must reorganize this expression into a linear system. We treat the scalar multiplication λv as matrix-vector multiplication using the identity matrix I:

$$Av - \lambda Iv = 0$$

$$(A - \lambda I)v = 0$$

For this equation to have a non-zero solution for v, the transformation $(A-\lambda I)$ must be "singular," meaning it squishes space into a lower dimension. This occurs only when the **determinant** is zero:

$$det(A-\lambda I)=0$$

Finding the "sweet spot" where this determinant is zero allows us to identify the eigenvectors that define our new coordinate system. In PCA, these vectors point toward the directions of the highest data "spread," or maximum variance.

PCA follows a rigid sequence to move from raw, unaligned data to a structured set of principal components. We will describe the steps with an example. Let's assume we have the following data:

Person	Age	Height (cm)	Weight (kg)
P1	25	175	70
P2	35	180	85
P3	45	170	75
P4	20	165	60

3.1 Step 1: Standardization:

Because features often have different units (e.g., age vs. salary), they must be scaled so that no single feature dominates the analysis simply due to its magnitude. Each feature is transformed to have a **mean of 0** and a **standard deviation of 1** using the Z-score normalization:

$$Z = \frac{X-\mu}{\sigma}$$

For all features : Calculate Mean (μ) and St. Deviation (σ)

Age = 31.25
Height = 172.5
Weight = 72.5

Age = 9.68
Height = 5.59
Weight = 9.35

Standardized Matrix (Z) =

Person	**Age (Std)**	**Height (Std)**	**Weight (Std)**
P1	-0.652	0.447	-0.279
P2	0.391	1.342	1.395
P3	1.434	-0.447	0.279
P4	-1.173	-1.342	-1.395

3.2 Step 2: Covariance Matrix Computation

PCA calculates a **covariance matrix** to identify how variables relate to one another. The covariance between two features x_1 and x_2 is:

$$\text{cov}(x_1, x_2) = \frac{\sum_{i=1}^{n} (x_{1i} - \overline{x_1})(x_{2i} - \overline{x_2})}{n-1}$$

A positive covariance indicates the variables increase together, while a negative value suggests an inverse relationship. The resulting matrix (e.g., 2×2 for two features) contains the variance of each feature on the diagonal and the covariances on the off-diagonals.

cov(Z) = (Transpose of Matrix Z x Standardized Matrix (Z)) / n-1

Where, n = samples/rows,

Transpose of Matrix, Z =

-0.652	0.391	1.434	-1.173
0.447	1.342	-0.447	-1.342
-0.279	1.395	0.279	-1.395

So, cov(z) = Matrix A (in the next step)

1.00	0.291	0.690
0.291	1.00	0.875
0.690	0.875	1.00

3.3 Step 3: Eigen-Decomposition

By applying $det(A-\lambda I) = 0$ to the covariance matrix, we extract the principal directions (eigenvectors) and their magnitudes (eigenvalues). These eigenvalues dictate which directions capture the most information. Where,

- **A**: The covariance matrix.
- **v**: The **eigenvector**, which defines the direction of a principal component.
- λ: The **eigenvalue**, a scalar that represents the magnitude or "importance" (variance) of that direction.
- I: Identity Matrix.

$det(A-\lambda I) = 0 =$

1.00-λ	0.291	0.690
0.291	**1.00-λ**	0.875
0.690	0.875	**1.00-λ**

Solutions for λ = EigenValue =

2.228	**= PC1**	**2.228 / 3.00 = 74.3 %**	**Of Data/info can be explained by PC1**
0.716	= PC2	0.716 / 3.00 = 23.87 %	Of Data/info can be explained by PC 2
0.056	= PC3	0.056 / 3.00 = 1.86 %	Of Data/info can be explained by PC 23

Decision: Keeping PC1 + PC2 will retain (74.3 + 23.87) = 98% (approx.) of all information of the dataset.

4 Geometric Intuition: Maximizing Variance

PCA seeks a new coordinate system where the first axis captures the maximal "spread" of the data. The Optimization Goal Mathematically, we seek a line that minimizes the sum of squared distances from the points to the line (the error). However, because the distance from any data point to the origin is fixed, the Pythagorean Theorem dictates that minimizing the perpendicular distance to the line is mathematically identical to maximizing the distance from the projected points to the origin. PCA therefore chooses the axis that maximizes this projected variance.

- **First Principal Component (PC1):** This is the line that minimizes the perpendicular distance from the data points to the line, or equivalently, **maximizes the variance** of the projected points.
- **Second Principal Component (PC2):** This is the next best direction that is **perpendicular (orthogonal)** to PC1.

The projections are calculated using dot products. For a data point vector P_1 and a unit vector μ representing the principal component, the projection P_1' is

$$P_1' = P_1 . \mu$$

This scalar value represents the distance of the projected point from the origin along the new axis.

5 PCA in Scikit-Learn: The SVD Algorithm

While eigendecomposition provides the conceptual blueprint, it is often numerically fragile. Scikit-Learn defaults to **Singular Value Decomposition (SVD)** because it is more stable and efficient for large datasets.

5.1 What is SVD?

SVD is a matrix factorization method that decomposes the input data matrix directly, rather than computing the covariance matrix first. Scikit-learn offers several solvers:

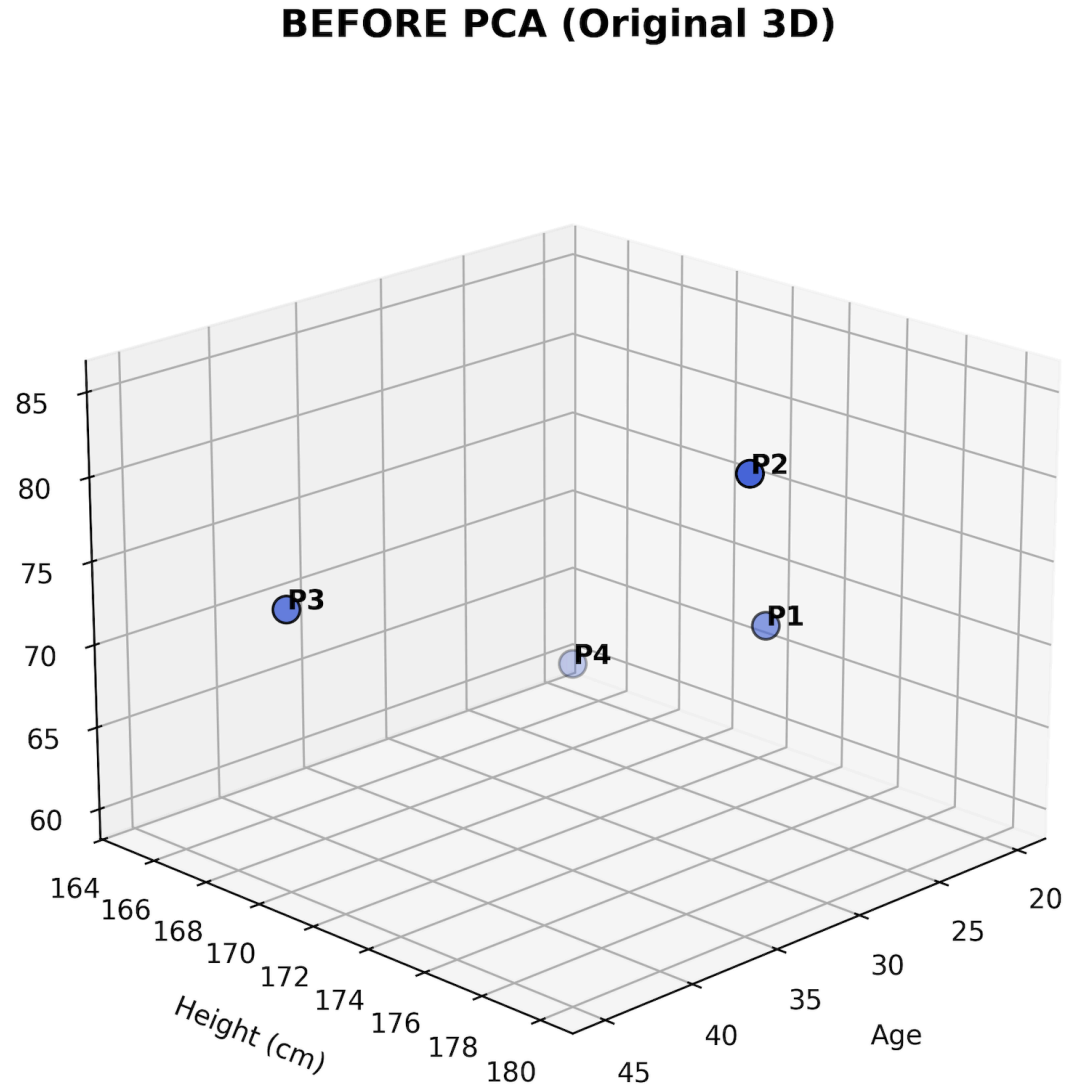
- **'full'**: Runs exact full SVD using LAPACK.
- **'arpack'**: Used for sparse data to find a truncated SVD.
- **'randomized'**: A fast, approximate SVD for large datasets where no of components is much smaller than the total features.

- **'covariance_eigh'**: Specifically efficient when the number of samples is much larger than the number of features.

5.2 Practical Implementation and Visualization (Using Scikit-Learn)

The typical workflow involves importing PCA from sklearn.decomposition, standardizing the data with StandardScaler, and then applying fit_transform.

Before PCA transformation, if we visualize the given example, it looks like the following:



Then, if we run PCA on the same example data through the scikit-learn library,

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

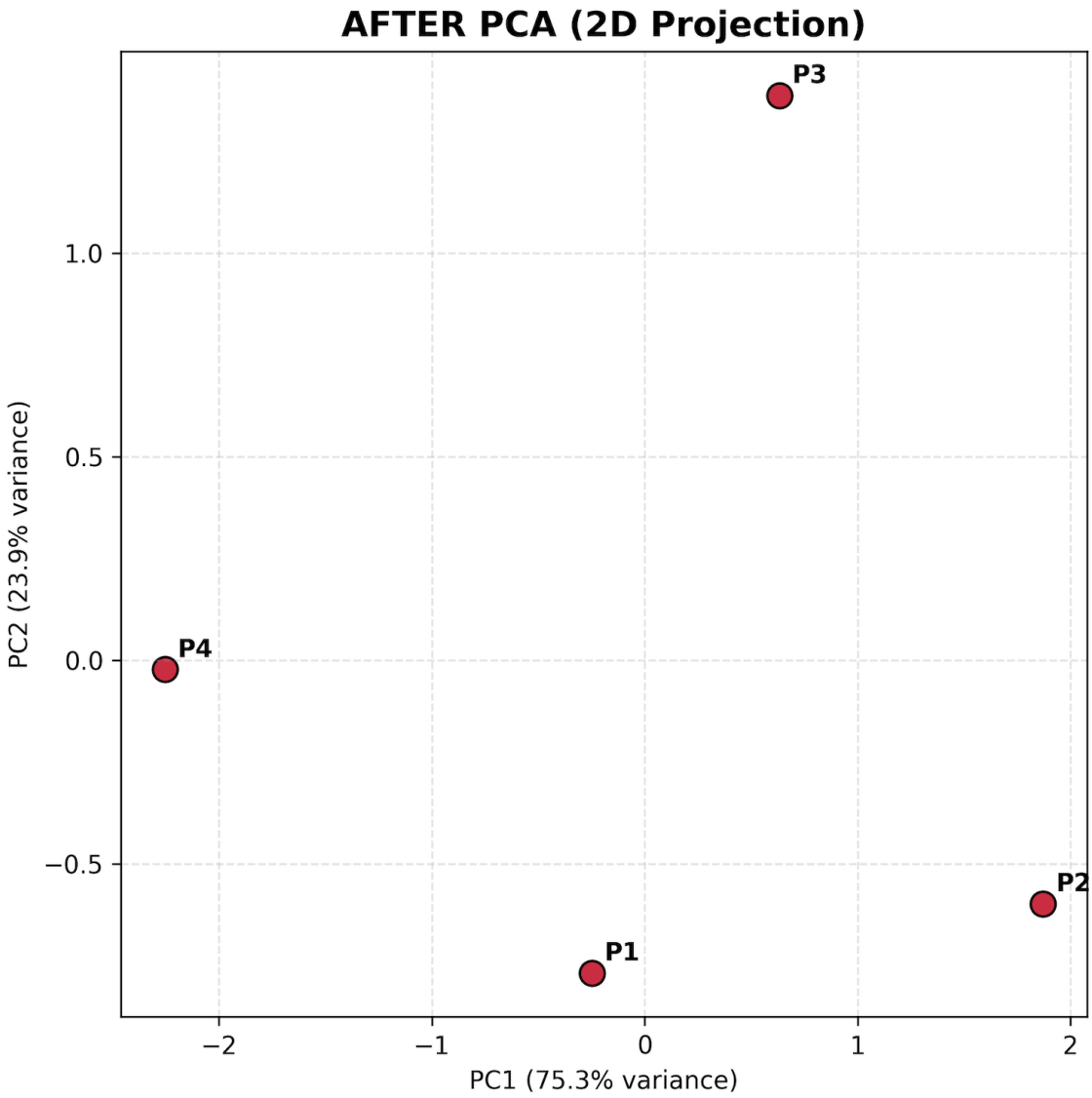
# 1. Create your custom dataset
data = {
   'Person': ['P1', 'P2', 'P3', 'P4'],
   'Age': [25, 35, 45, 20],
   'Height': [175, 180, 170, 165],
   'Weight': [70, 85, 75, 60]
}

df = pd.DataFrame(data)
# Select the numeric features for PCA
features = ['Age', 'Height', 'Weight']
X = df[features]

# 2. Standardize the data (Mean=0, Variance=1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Apply PCA to reduce from 3D to 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

And then visualize the data with Principal Component 1 (PC1) and Principal Component 2 (PC2); it looks like the following:



Before and After PCA Analysis:

- **Before PCA:** A scatter plot of raw standardized features shows there can be multiple ways to separate the data points (it would be more visible with lots of data points).
- **After PCA:** When plotted using **PC1 and PC2**, the data points show much clearer **class separation and structure**. This is because PC1 captures the "essence" of the data by prioritizing the directions of highest spread.

Appendix: Few Sample Results

Baseline: Wine Datasets (13 Features, 178 Observations)
PCA: 9 Features (Dropped 4 Features, Dataset Size Reduced 30%)

Classifier	Baseline	PCA Accuracy (95% Var)	Difference
Random Forest	99.2% ± 0.01	98.5% ± 0.02	-0.7%
SVM	98.1% ± 0.02	98.0% ± 0.02	~
Multilayer Perceptron	97.5% ± 0.03	98.2% ± 0.02	+0.7% Improve

The table shows that even after dropping 4 features, the accuracy loss was not that much; in some cases (MLP), the accuracy was even better.